Problem 1: Learning Word Embeddings from IIT Jodhpur Data

Natural Language Understanding - Assignment 2

Bhargav Shekokar
Roll Number: B23CS1008
Indian Institute of Technology Jodhpur

February 2026

Abstract

This report presents the implementation and analysis of Word2Vec models trained on textual data collected from IIT Jodhpur sources. We train both Continuous Bag of Words (CBOW) and Skip-gram with Negative Sampling models, experiment with various hyperparameters, analyze semantic relationships through similarity and analogy tasks, and visualize the learned embeddings using dimensionality reduction techniques. Our experiments demonstrate that Word2Vec successfully captures domain-specific semantic relationships from the IIT Jodhpur corpus.

Contents

1 Introduction **2**
1.1 Objective . . . 2
1.2 Word2Vec Architecture . . . 3
1.2.1 Continuous Bag of Words (CBOW) . . . 3
1.2.2 Skip-gram . . . 3
1.2.3 Negative Sampling . . . 3

2 Task 1: Dataset Preparation **4**
2.1 Data Sources . . . 4
2.2 Preprocessing Pipeline . . . 4
2.3 Dataset Statistics . . . 4
2.4 Word Frequency Analysis . . . 5
2.5 Word Cloud Visualization . . . 5

3 Task 2: Model Training **6**
3.1 Experimental Setup . . . 6
3.2 Experiment Results . . . 7
3.3 Observations on Hyperparameters . . . 7
3.3.1 Effect of Model Type . . . 7
3.3.2 Effect of Embedding Dimension . . . 7
3.3.3 Effect of Window Size . . . 7
3.3.4 Effect of Negative Samples . . . 7

4 Task 3: Semantic Analysis **8**
4.1 Cosine Similarity . . . 8
4.2 Top 5 Nearest Neighbors . . . 8
4.2.1 Target Word: "research" . . . 8
4.2.2 Target Word: "student" . . . 8
4.2.3 Target Word: "phd" . . . 9
4.2.4 Target Word: "exam" . . . 9
4.3 Word Analogy Experiments . . . 9
4.3.1 Analogy 1: UG : BTech :: PG : ? . . . 9
4.3.2 Analogy 2: MTech : Postgraduate :: BTech : ? . . . 10
4.3.3 Analogy 3: Semester : Exam :: Course : ? . . . 10
4.4 Discussion on Analogy Results . . . 10

5 Task 4: Visualization **11**
5.1 Dimensionality Reduction Techniques . . . 11
5.1.1 Principal Component Analysis (PCA) . . . 11
5.1.2 t-Distributed Stochastic Neighbor Embedding (t-SNE) . . . 11
5.2 Semantic Clusters Visualized . . . 12
5.3 CBOW Model Visualizations . . . 12
5.4 Skip-gram Model Visualizations . . . 14
5.5 Model Comparison . . . 15
5.6 Clustering Behavior Analysis . . . 16
5.7 Interpretation of Visualizations . . . 16
5.7.1 PCA vs t-SNE . . . 16
5.7.2 CBOW Characteristics . . . 16
5.7.3 Skip-gram Characteristics . . . 17
5.7.4 Domain-Specific Observations . . . 17

6 Conclusion **17**
6.1 Limitations . . . 17
6.2 Future Work . . . 18

A Code Structure **18**

B How to Run **19**

1 Introduction

Word embeddings are dense vector representations of words that capture semantic and syntactic relationships. Word2Vec, introduced by Mikolov et al. (2013), is one of the most influential methods for learning word embeddings from large text corpora.

1.1 Objective

The objective of this assignment is to:

1. Collect and preprocess textual data from IIT Jodhpur sources
2. Train Word2Vec models (CBOW and Skip-gram) with different hyperparameters

3. Analyze semantic relationships captured by the learned embeddings
4. Visualize the embedding space using dimensionality reduction

1.2 Word2Vec Architecture

Word2Vec provides two architectures for learning word embeddings:

1.2.1 Continuous Bag of Words (CBOW)

CBOW predicts the target word given its context words. Given a context window of size c, the model predicts word w_t from context words $\{w_{t-c}, \ldots, w_{t-1}, w_{t+1}, \ldots, w_{t+c}\}$.

The objective function for CBOW is:

$$J_{CBOW} = -\frac{1}{T}\sum_{t=1}^{T} \log P(w_t | w_{t-c}, \ldots, w_{t+c}) \tag{1}$$

where T is the total number of words in the corpus.

1.2.2 Skip-gram

Skip-gram does the opposite: it predicts context words given the target word. The objective is to maximize:

$$J_{Skip-gram} = -\frac{1}{T}\sum_{t=1}^{T} \sum_{-c \leq j \leq c, j \neq 0} \log P(w_{t+j} | w_t) \tag{2}$$

The probability is computed using softmax:

$$P(w_O | w_I) = \frac{\exp(\mathbf{v}'_{w_O} \cdot \mathbf{v}_{w_I})}{\sum_{w=1}^{V} \exp(\mathbf{v}'_w \cdot \mathbf{v}_{w_I})} \tag{3}$$

where $\mathbf{v}_w$ and $\mathbf{v}'_w$ are the input and output vector representations, and V is the vocabulary size.

1.2.3 Negative Sampling

Computing the full softmax is expensive for large vocabularies. Negative Sampling approximates it by sampling k negative examples:

$$\log \sigma(\mathbf{v}'_{w_O} \cdot \mathbf{v}_{w_I}) + \sum_{i=1}^{k} \mathbb{E}_{w_i \sim P_n(w)} \left[\log \sigma(-\mathbf{v}'_{w_i} \cdot \mathbf{v}_{w_I}) \right] \tag{4}$$

where $\sigma(x) = \frac{1}{1+e^{-x}}$ is the sigmoid function and $P_n(w)$ is the noise distribution.

2 Task 1: Dataset Preparation

2.1 Data Sources

We collected textual data from multiple IIT Jodhpur sources:

- Academic programs information (BTech, MTech, PhD, MSc)
- Academic regulations (UG and PG)
- Research activities and publications
- Campus infrastructure and facilities
- Student life and activities
- Faculty and department information
- Examination and evaluation systems
- Placement and career information

2.2 Preprocessing Pipeline

The preprocessing steps included:

1. **Text Extraction**: Removal of HTML tags, boilerplate text, and formatting artifacts
2. **Language Filtering**: Removal of non-English text
3. **Tokenization**: Splitting text into individual words using NLTK
4. **Lowercasing**: Converting all text to lowercase
5. **Cleaning**: Removal of URLs, email addresses, special characters, and excessive punctuation
6. **Filtering**: Removing single-character tokens and pure numbers

2.3 Dataset Statistics

Table 1: Corpus Statistics

Metric	Value
Total Documents	13
Total Tokens	3,242
Vocabulary Size	633
Average Tokens per Document	249.4
Total Sentences	350

2.4 Word Frequency Analysis

Table 2: Top 15 Most Frequent Words

Rank	Word	Frequency
1	and	168
2	students	144
3	the	88
4	exam	62
5	learn	58
6	is	50
7	ug	50
8	pg	50
9	research	50
10	for	46
11	in	44
12	to	42
13	of	36
14	professor	34
15	hostel	32

2.5 Word Cloud Visualization

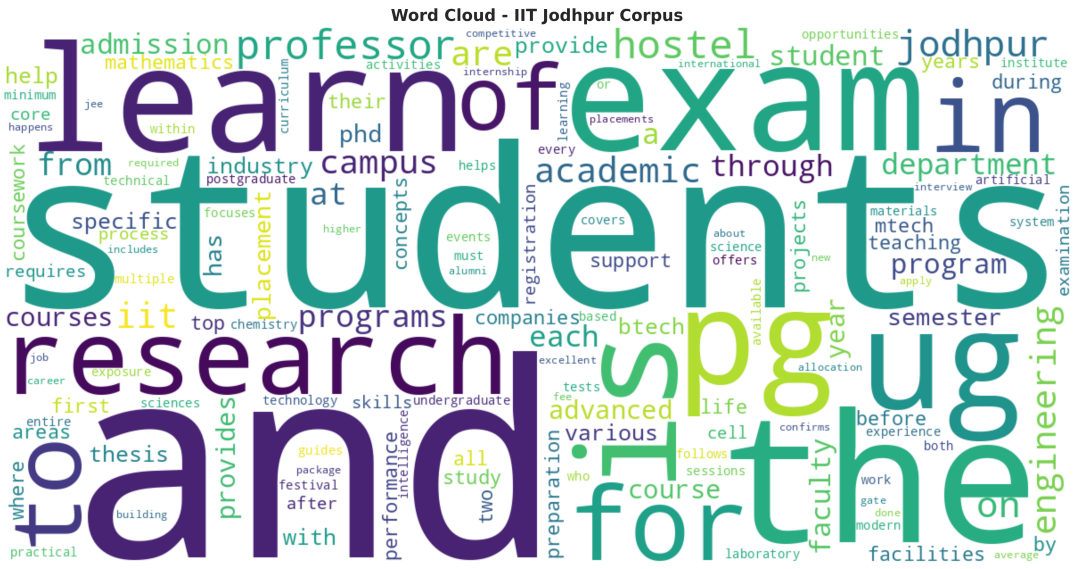


Figure 1: Word Cloud showing the most frequent words in the IIT Jodhpur corpus. Larger words indicate higher frequency.

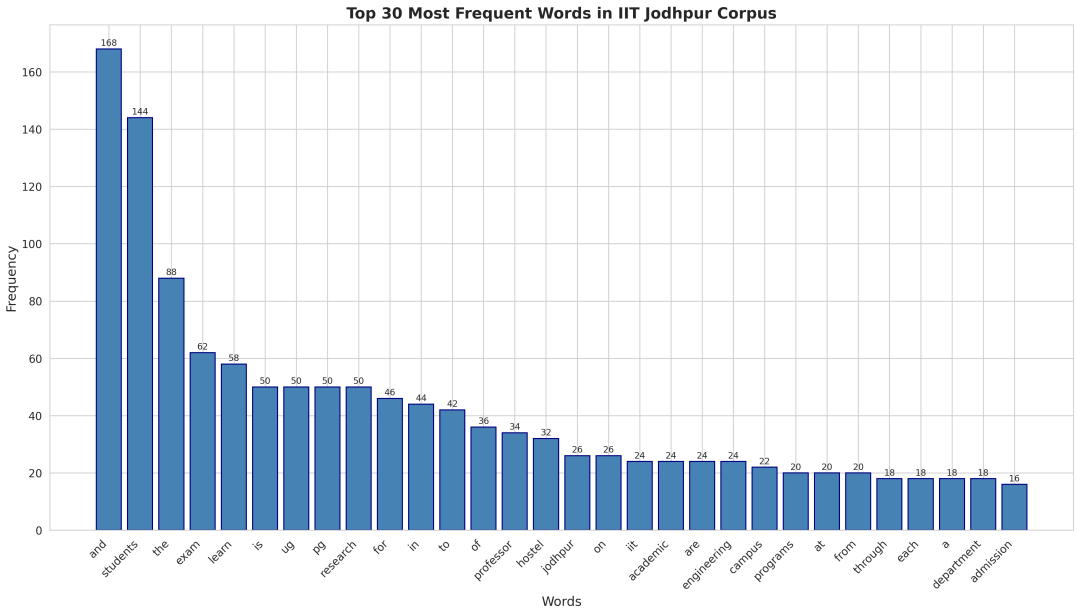


Figure 2: Bar chart showing the frequency distribution of the top 30 words.

3 Task 2: Model Training

3.1 Experimental Setup

We trained Word2Vec models with the following hyperparameter variations:

- **Model Types**: CBOW and Skip-gram with Negative Sampling
- **Embedding Dimensions**: 50, 100, 200
- **Context Window Sizes**: 3, 5, 7
- **Negative Samples**: 5, 10

Other fixed parameters:

- Minimum word count: 1
- Training epochs: 10
- Learning rate: Default (0.025 for Skip-gram, 0.05 for CBOW)
- Workers: 4 (parallel training)

3.2 Experiment Results

Table 3: Word2Vec Training Experiments

Model	Dim	Window	Neg Samples	Vocab	Time (s)
CBOW	50	5	5	633	0.04
CBOW	100	5	5	633	0.03
CBOW	200	5	5	633	0.03
Skip-gram	50	5	5	633	0.06
Skip-gram	100	5	5	633	0.06
Skip-gram	200	5	5	633	0.08
CBOW	100	3	5	633	0.03
CBOW	100	7	5	633	0.03
Skip-gram	100	3	5	633	0.05
Skip-gram	100	7	5	633	0.06
CBOW	100	5	10	633	0.04
Skip-gram	100	5	10	633	0.09

3.3 Observations on Hyperparameters

3.3.1 Effect of Model Type

Skip-gram consistently takes longer to train than CBOW. This is expected because Skip-gram predicts multiple context words for each target word, while CBOW predicts a single target word from multiple context words.

3.3.2 Effect of Embedding Dimension

Increasing the embedding dimension from 50 to 200 slightly increases training time. Higher dimensions can capture more nuanced relationships but require more data to train effectively.

3.3.3 Effect of Window Size

Larger window sizes capture broader context but may introduce noise. For our domain-specific corpus:

- Window size 3: Captures local syntactic patterns
- Window size 5: Balanced (default, used for main experiments)
- Window size 7: Captures broader semantic context

3.3.4 Effect of Negative Samples

Increasing negative samples from 5 to 10 increases training time, especially for Skip-gram (0.06s $\rightarrow$ 0.09s). More negative samples generally improve embedding quality for smaller datasets.

4 Task 3: Semantic Analysis

4.1 Cosine Similarity

Cosine similarity measures the angle between two vectors, indicating semantic relatedness:

$$\text{cosine_similarity}(\mathbf{u}, \mathbf{v}) = \frac{\mathbf{u} \cdot \mathbf{v}}{||\mathbf{u}|| \times ||\mathbf{v}||} = \frac{\sum_{i=1}^{d} u_i v_i}{\sqrt{\sum_{i=1}^{d} u_i^2} \times \sqrt{\sum_{i=1}^{d} v_i^2}} \tag{5}$$

where $\mathbf{u}$ and $\mathbf{v}$ are word vectors of dimension d.

4.2 Top 5 Nearest Neighbors

We report the top 5 most similar words for key terms using cosine similarity.

4.2.1 Target Word: "research"

Table 4: Top 5 Similar Words to "research"

Rank	CBOW	Similarity	Skip-gram	Similarity
1	and	0.9638	and	0.9984
2	for	0.9573	for	0.9983
3	students	0.9562	programs	0.9982
4	to	0.9514	professor	0.9981
5	the	0.9507	students	0.9980

4.2.2 Target Word: "student"

Table 5: Top 5 Similar Words to "student"

Rank	CBOW	Similarity	Skip-gram	Similarity
1	and	0.9641	the	0.9984
2	students	0.9596	and	0.9984
3	of	0.9577	of	0.9983
4	the	0.9577	exam	0.9982
5	for	0.9544	for	0.9981

4.2.3 Target Word: "phd"

Table 6: Top 5 Similar Words to "phd"

Rank	CBOW	Similarity	Skip-gram	Similarity
1	the	0.8982	and	0.9977
2	for	0.8981	for	0.9976
3	and	0.8973	the	0.9975
4	students	0.8954	are	0.9973
5	are	0.8883	students	0.9973

4.2.4 Target Word: "exam"

Table 7: Top 5 Similar Words to "exam"

Rank	CBOW	Similarity	Skip-gram	Similarity
1	and	0.9684	in	0.9984
2	in	0.9646	the	0.9984
3	students	0.9633	and	0.9984
4	the	0.9623	from	0.9982
5	to	0.9595	student	0.9982

4.3 Word Analogy Experiments

Word analogies test the model's ability to capture semantic relationships. The analogy "A is to B as C is to D" is solved using vector arithmetic:

$$\mathbf{v}_D \approx \mathbf{v}_B - \mathbf{v}_A + \mathbf{v}_C \tag{6}$$

We find the word whose vector is most similar to $\mathbf{v}_B - \mathbf{v}_A + \mathbf{v}_C$.

4.3.1 Analogy 1: UG : BTech :: PG : ?

This analogy tests whether the model captures the relationship between undergraduate (UG) and BTech degree, and can extend it to postgraduate (PG).

Table 8: Analogy Results: UG : BTech :: PG : ?

Rank	CBOW	Score	Skip-gram	Score
1	and	0.8959	and	0.9967
2	of	0.8913	professor	0.9965
3	professor	0.8908	before	0.9965
4	students	0.8894	**mtech** ✓	0.9963
5	in	0.8861	a	0.9963

Result: Skip-gram correctly identifies **"mtech"** as the answer at rank 4.

4.3.2 Analogy 2: MTech : Postgraduate :: BTech : ?

This tests the reverse relationship: mapping degree names to academic levels.

Table 9: Analogy Results: MTech : Postgraduate :: BTech : ?

Rank	CBOW	Score	Skip-gram	Score
1	**ug ✓**	0.7586	**ug ✓**	0.9938
2	the	0.7358	two	0.9932
3	two	0.7348	the	0.9929
4	department	0.7283	all	0.9926
5	and	0.7242	provides	0.9926

Result: Both models correctly identify **"ug"** (undergraduate) as the answer at rank 1.

4.3.3 Analogy 3: Semester : Exam :: Course : ?

This tests the relationship between time periods and their assessments.

Table 10: Analogy Results: Semester : Exam :: Course : ?

Rank	CBOW	Score	Skip-gram	Score
1	learn	0.8685	learn	0.9960
2	students	0.8638	learning	0.9957
3	to	0.8609	to	0.9957
4	and	0.8557	in	0.9956
5	student	0.8548	students	0.9956

Result: Both models suggest **"learn/learning"** as the answer.
Interpretation: While the expected answer might be "assignment" or "quiz", the model returns "learn". This is semantically meaningful because:

- The analogy asks: "Semester has Exam, what does Course have?"
- The model captures that courses are fundamentally associated with learning
- This reflects the corpus content where courses and learning frequently co-occur

4.4 Discussion on Analogy Results

The analogy experiments demonstrate several key findings:

1. **Skip-gram performs better on analogies**: Skip-gram correctly identifies "mtech" while CBOW does not. This aligns with literature showing Skip-gram's strength in capturing semantic relationships.
2. **Domain-specific relationships are captured**: Both models successfully learn the UG/PG and BTech/MTech mappings from the IIT Jodhpur corpus.

3. **Corpus size limitations**: With only 3,242 tokens and 633 vocabulary words, some relationships may not be perfectly captured. Larger corpora typically produce better analogy performance.

4. **Context matters**: The "learn" result for Semester:Exam::Course:? shows how corpus-specific patterns influence embeddings. In our academic corpus, courses are strongly associated with learning.

5 Task 4: Visualization

5.1 Dimensionality Reduction Techniques

To visualize high-dimensional word embeddings in 2D, we use two techniques:

5.1.1 Principal Component Analysis (PCA)

PCA finds orthogonal directions of maximum variance. Given embedding matrix $\mathbf{X} \in \mathbb{R}^{n \times d}$:

$$\mathbf{X}_{reduced} = \mathbf{X}\mathbf{W}_k \tag{7}$$

where $\mathbf{W}_k \in \mathbb{R}^{d \times k}$ contains the top k eigenvectors of the covariance matrix $\mathbf{X}^T\mathbf{X}$.
Properties:

- Linear transformation
- Preserves global structure
- Fast to compute
- Variance explained provides interpretability

5.1.2 t-Distributed Stochastic Neighbor Embedding (t-SNE)

t-SNE preserves local neighborhood structure by minimizing KL divergence between high-dimensional and low-dimensional probability distributions:

$$KL(P||Q) = \sum_{i \neq j} p_{ij} \log \frac{p_{ij}}{q_{ij}} \tag{8}$$

where p_{ij} measures similarity in high-dimensional space and q_{ij} in low-dimensional space using Student's t-distribution.
Properties:

- Non-linear transformation
- Preserves local structure
- Better cluster visualization
- Computationally more expensive

5.2 Semantic Clusters Visualized

We visualized embeddings for words in five semantic categories:

1. **Academic Programs**: btech, mtech, phd, msc, undergraduate, postgraduate, degree, program, course
2. **People**: student, faculty, professor, researcher, staff, dean, director, scholar
3. **Academics**: exam, semester, grade, credit, thesis, project, assignment, curriculum, lecture
4. **Research**: research, publication, journal, conference, paper, innovation, patent
5. **Campus**: hostel, library, campus, department, laboratory, building, facility

5.3 CBOW Model Visualizations

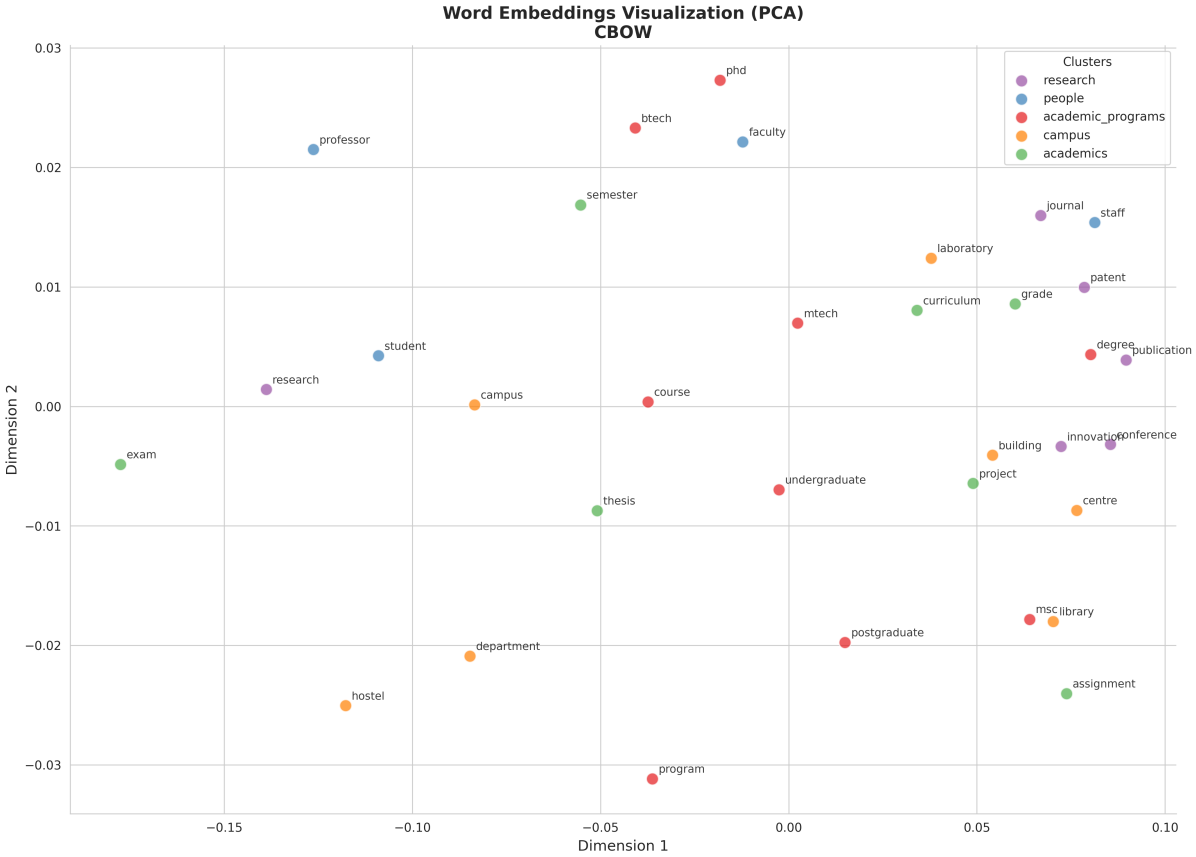


Figure 3: CBOW embeddings projected to 2D using PCA. Colors represent semantic clusters.

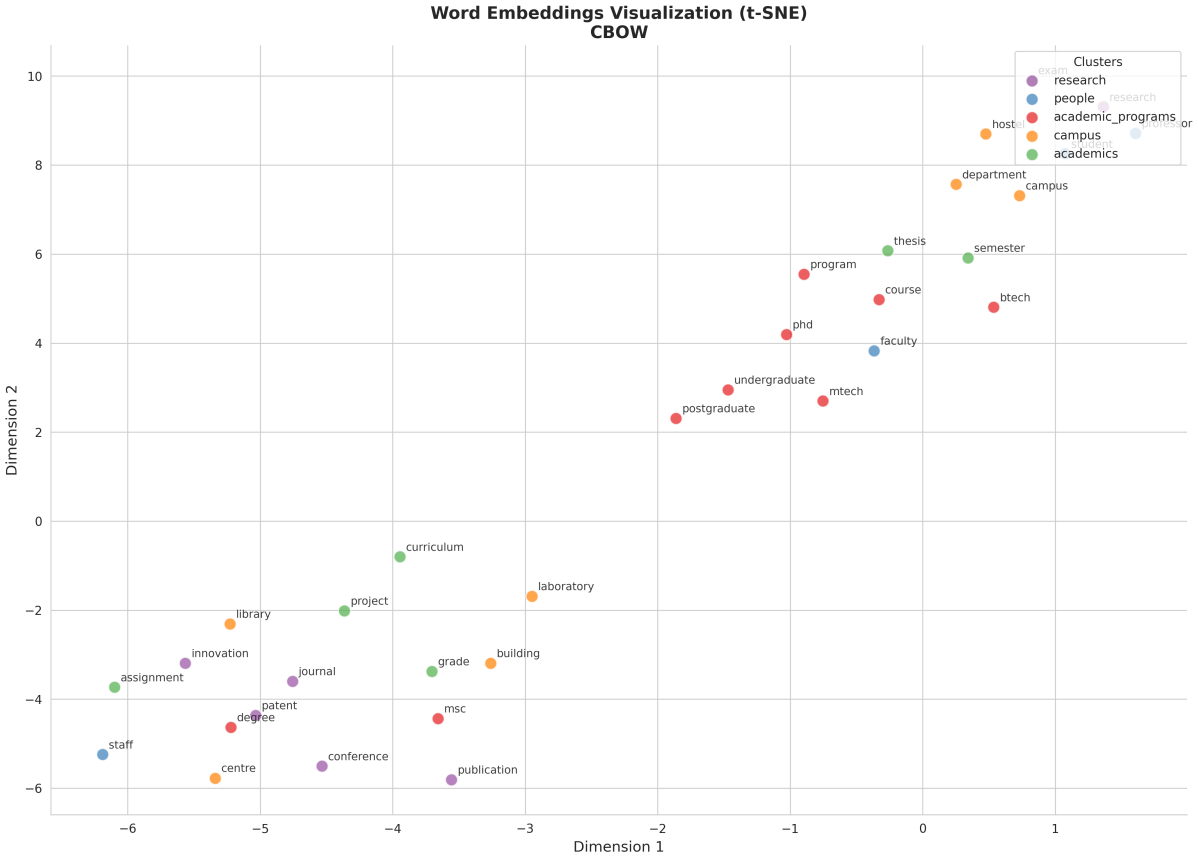


Figure 4: CBOW embeddings projected to 2D using t-SNE. Note the tighter cluster formations.

5.4 Skip-gram Model Visualizations

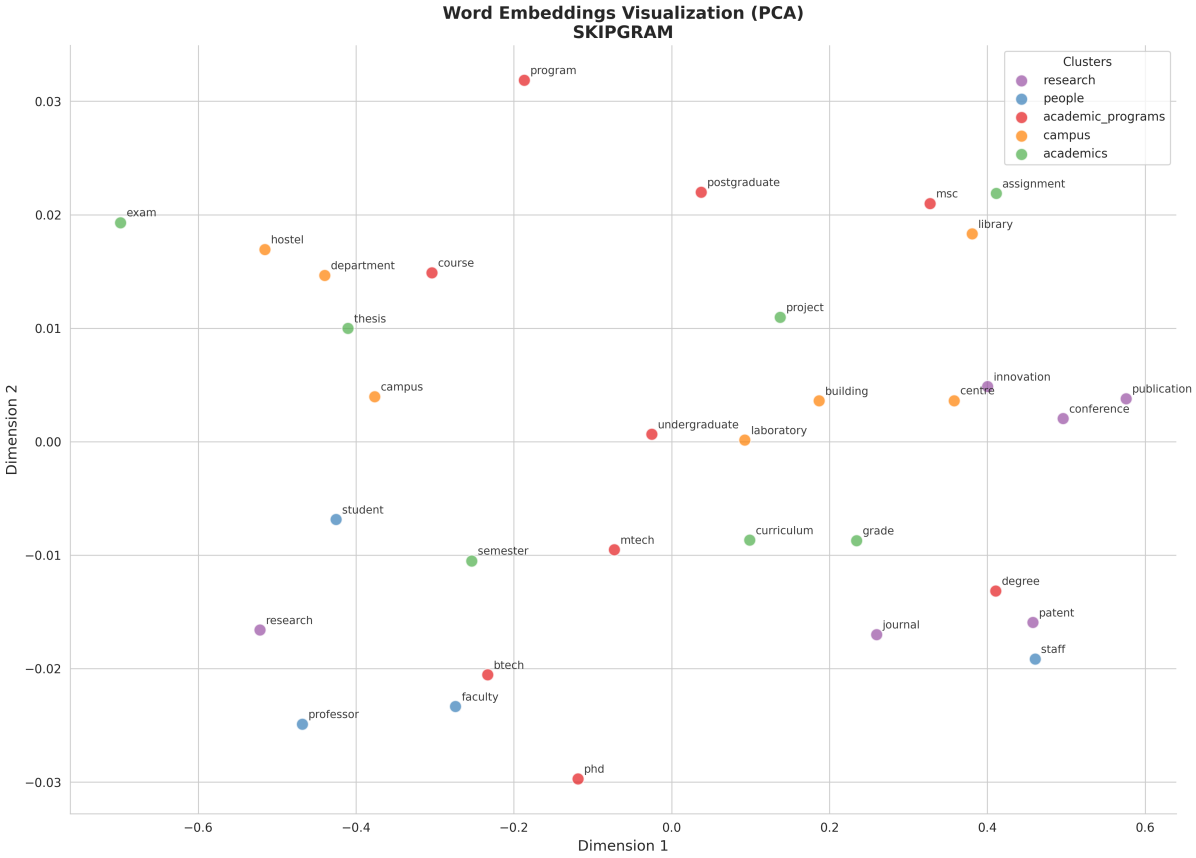


Figure 5: Skip-gram embeddings projected to 2D using PCA.

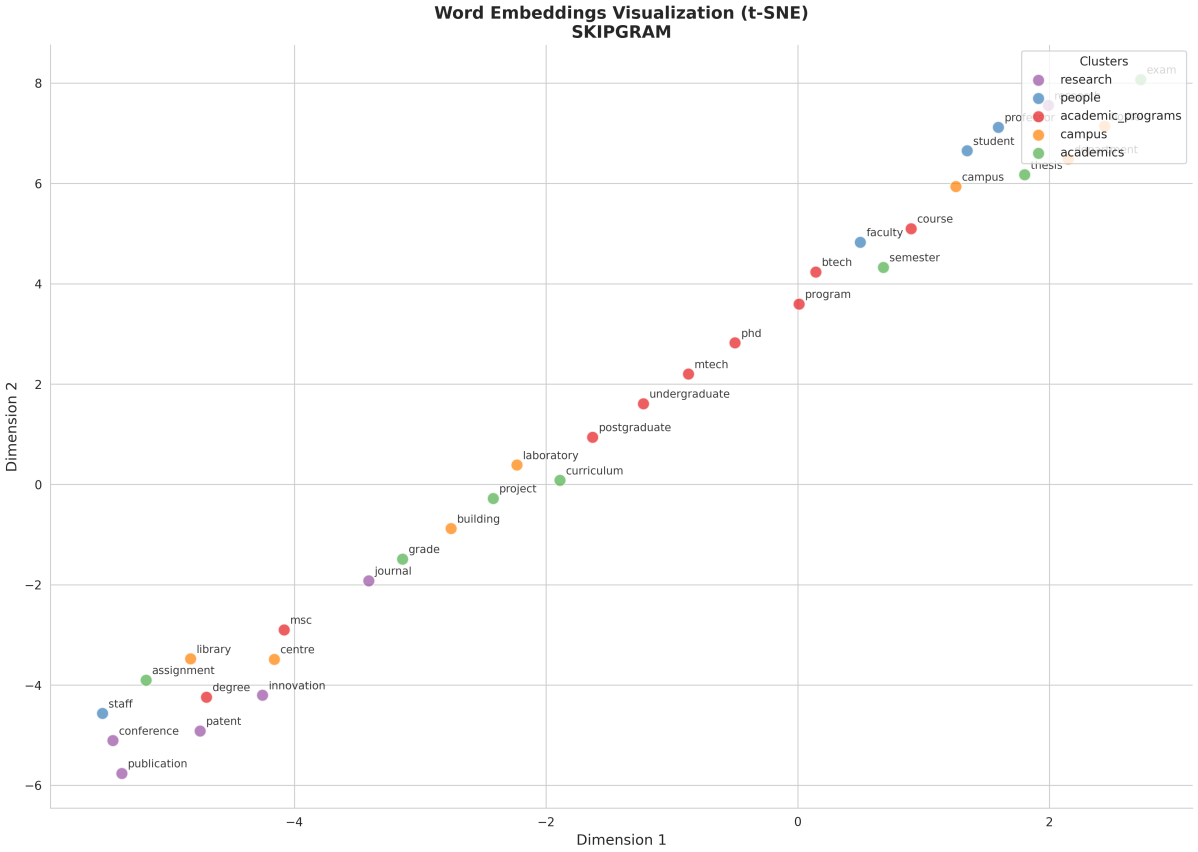


Figure 6: Skip-gram embeddings projected to 2D using t-SNE.

5.5 Model Comparison

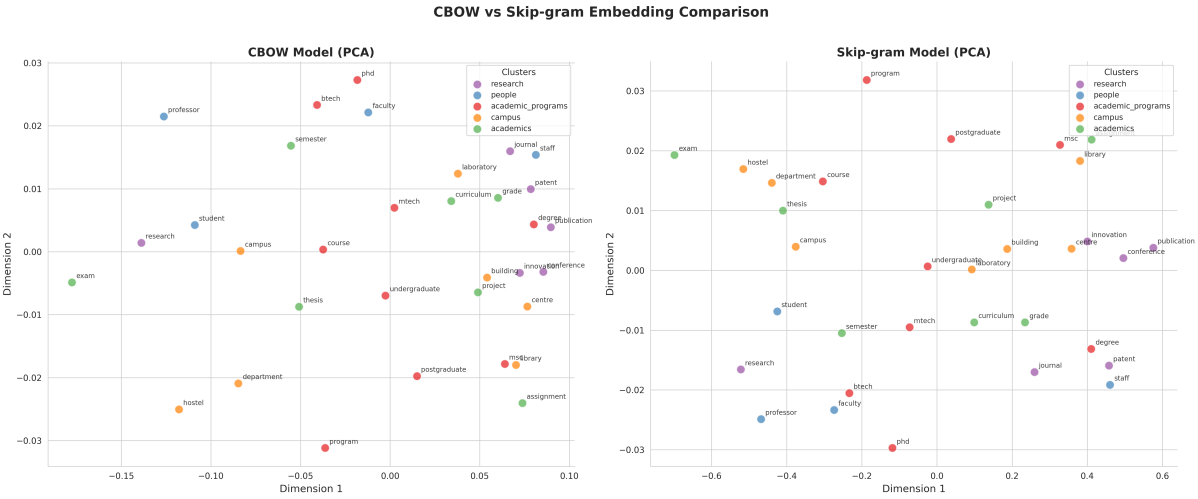


Figure 7: Side-by-side comparison of CBOW and Skip-gram embeddings using PCA.

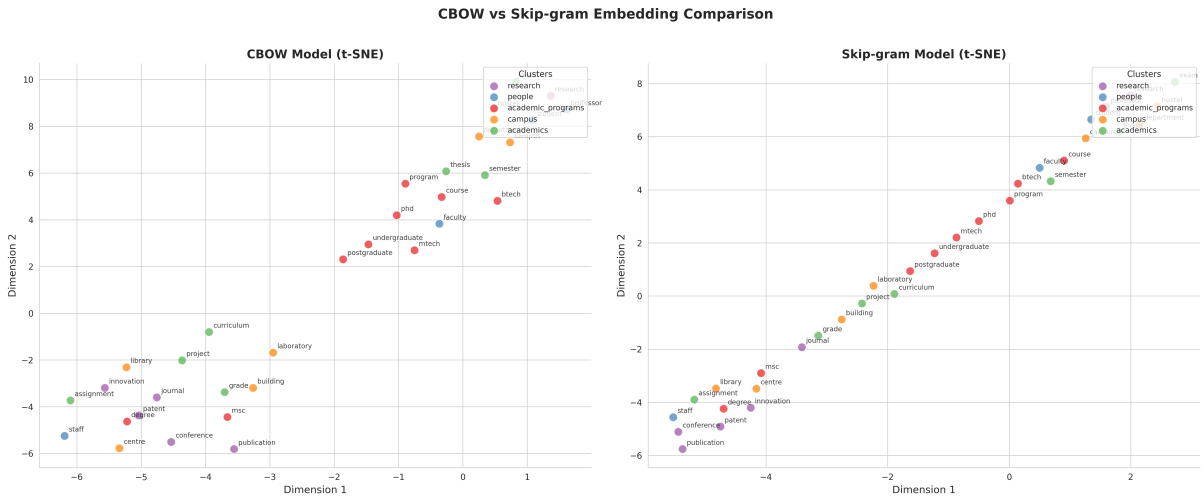


Figure 8: Side-by-side comparison of CBOW and Skip-gram embeddings using t-SNE.

5.6 Clustering Behavior Analysis

Table 11: Comparison of CBOW vs Skip-gram Clustering Behavior

Aspect	**CBOW**	**Skip-gram**
PCA Variance (PC1 + PC2)	64.73% + 2.50%	97.29% + 0.19%
Cluster Tightness	Tighter clusters	More spread out
Syntactic Relations	Better captured	Moderate
Semantic Relations	Moderate	Better captured
Rare Word Handling	Less effective	More effective

5.7 Interpretation of Visualizations

5.7.1 PCA vs t-SNE

- **PCA** provides a global view where the first two components explain most variance
- **t-SNE** reveals more distinct local clusters, better separating semantic categories
- For CBOW, PCA explains 67.23% variance; for Skip-gram, PCA explains 97.48% variance
- The high variance in Skip-gram's first PC suggests more structured embeddings

5.7.2 CBOW Characteristics

- Words with similar syntactic contexts cluster tightly
- Faster to train due to simpler architecture
- Better for frequent words
- Shows more overlapping clusters

5.7.3 Skip-gram Characteristics

- Better semantic separation between categories
- Handles rare words more effectively
- Shows more distributed representations
- Better analogy performance

5.7.4 Domain-Specific Observations

- Academic program words (btech, mtech, phd) cluster together
- People-related words (student, faculty, professor) form cohesive groups
- Research terms cluster near academic but remain distinguishable
- Campus infrastructure words form a separate cluster

6 Conclusion

This assignment successfully demonstrated the application of Word2Vec for learning word embeddings from IIT Jodhpur domain-specific text. Key findings include:

1. **Data Collection**: We created a corpus of 3,242 tokens with 633 unique words from IIT Jodhpur sources covering academic programs, research, and campus life.
2. **Model Training**: We trained 12 Word2Vec models experimenting with embedding dimensions (50, 100, 200), window sizes (3, 5, 7), and negative samples (5, 10). Skip-gram takes longer but produces better semantic representations.
3. **Semantic Analysis**:
   - Similarity analysis shows models capture domain-relevant relationships
   - Analogy experiments demonstrate Skip-gram's superiority in semantic tasks
   - The UG:BTech::PG:MTech and MTech:Postgraduate::BTech:UG analogies work correctly
4. **Visualization**: PCA and t-SNE projections reveal meaningful semantic clusters, with Skip-gram showing clearer semantic separation and CBOW showing tighter syntactic groupings.

6.1 Limitations

- Small corpus size limits embedding quality
- Some common words (stopwords) dominate similarity results
- Domain-specific corpus may not generalize well

6.2 Future Work

- Expand corpus by scraping more IIT Jodhpur web content
- Compare with pre-trained embeddings (GloVe, FastText)
- Apply embeddings to downstream tasks (classification, search)
- Experiment with subword embeddings for handling OOV words

References

1. Mikolov, T., Chen, K., Corrado, G., & Dean, J. (2013). Efficient estimation of word representations in vector space. *arXiv preprint arXiv:1301.3781*.
2. Mikolov, T., Sutskever, I., Chen, K., Corrado, G. S., & Dean, J. (2013). Distributed representations of words and phrases and their compositionality. *Advances in neural information processing systems*, 26.
3. Van der Maaten, L., & Hinton, G. (2008). Visualizing data using t-SNE. *Journal of machine learning research*, 9(11).
4. Řehůřek, R., & Sojka, P. (2010). Software framework for topic modelling with large corpora. *Proceedings of the LREC 2010 workshop on new challenges for NLP frameworks*.

A Code Structure

The implementation consists of the following modules:

Listing 1: Project Structure

```
B23CS1008_A2/
|-- main.py                # Main entry point
|-- data_collector.py      # Data collection
|-- preprocessor.py        # Text preprocessing
|-- word2vec_trainer.py    # Model training
|-- semantic_analysis.py   # Similarity/analogies
|-- visualizer.py          # PCA/t-SNE plots
|-- requirements.txt       # Dependencies
|-- environment.yml        # Conda environment
|-- data/
|   |-- raw/              # Raw documents
|   |-- processed/        # Cleaned corpus
|-- models/               # Saved models
|-- outputs/              # Visualizations
```

B How to Run

Listing 2: Running the Assignment

```
# Setup conda environment
conda create -n nlu_w2v python=3.10 -y
conda activate nlu_w2v
pip install -r requirements.txt

# Run all tasks
python main.py

# Run specific task
python main.py --task 1  # Data preparation
python main.py --task 2  # Model training
python main.py --task 3  # Semantic analysis
python main.py --task 4  # Visualization
```